Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Teste de Software - PUC Minas (Prof. Cleiton Tavares)
Aluno: Rafael de Oliveira Caldeira Lopes
Matrícula: 829981 **Ferramentas:** Jest e StrykerJS
Repositório: https://github.com/RafaelLopes1810/operacoes-mutante.git

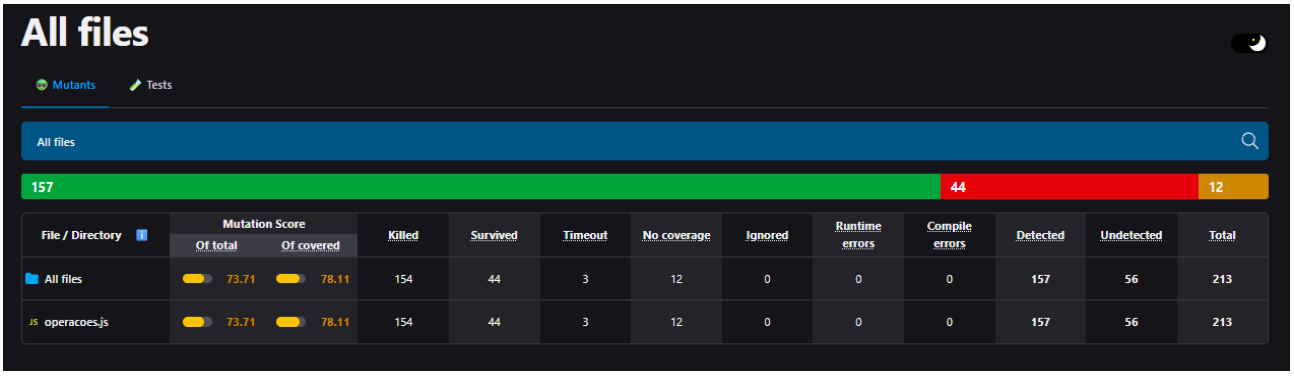
1. Análise inicial

O projeto é uma biblioteca com 50 funções matemáticas em src/operacoes.js. A suíte original tinha 50 testes, quase sempre um único exemplo "feliz" por função. Todos passavam e, olhando só a cobertura, parecia tudo ótimo. O Stryker contou outra história:

Métrica inicial	Resultado
Cobertura de statements / branches / functions / lines	85,41% / 58,82% / 100% / 98,64%
Mutation score (Stryker)	73,71%
Mutantes gerados	213
Mortos / timeouts / sobreviventes / sem cobertura	154 / 3 / 44 / 12

A diferença faz sentido: a cobertura só diz se a linha foi executada, não se algum teste perceberia um defeito ali. Chamar isMaiorQue(10, 5) executa a função, mas não distingue a > b de a >= b. Do mesmo jeito, um toThrow() sem mensagem aceita qualquer erro, até o de um mutante que troca o texto por uma string vazia. Ou seja, o teste passava por cima do código sem realmente conferi-lo.

Figura 1 - Resumo do relatório inicial do Stryker.



2. Análise de mutantes críticos

Dos 44 sobreviventes da primeira execução, escolhi três de tipos diferentes (a lista completa, com a classificação de cada um, está no apêndice mutantes-iniciais.md do repositório).

Local	Mutação feita pelo Stryker	Por que sobreviveu
operacoes.js:75 (isPrimo)	**BlockStatement**: removeu o if (n % i === 0) return false	O único teste era isPrimo(7), um primo. Nenhum número composto passava pelo ramo que rejeita divisores.
operacoes.js:44 (isImpar)	**ArithmeticOperator**: n % 2 !== 0 virou n * 2 !== 0	Para n = 7 as duas expressões dão true. Faltava um número par para mostrar a diferença.
operacoes.js:104 (isMaiorQue)	**EqualityOperator**: a > b virou a >= b	Só havia valores diferentes com a > b. Sem testar a === b (a fronteira), os dois operadores dão o mesmo resultado.

Figura 2 - Mutantes sobrevivente em isPrimo.

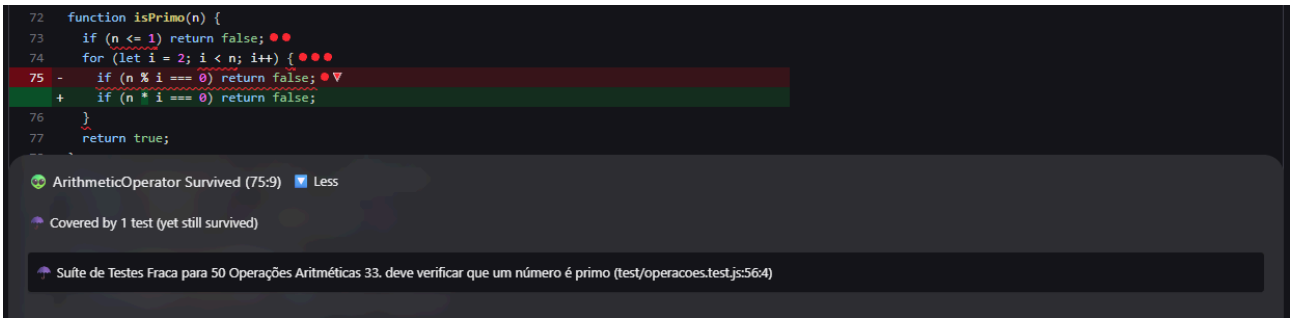


Figura 3 - Mutante sobrevivente em isImpar.

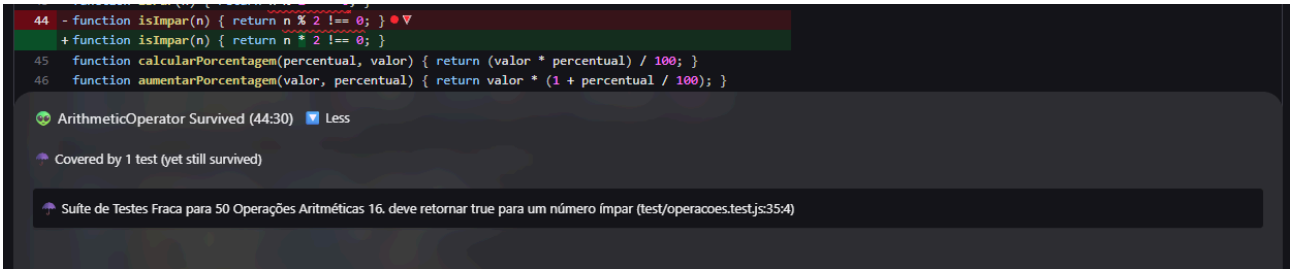
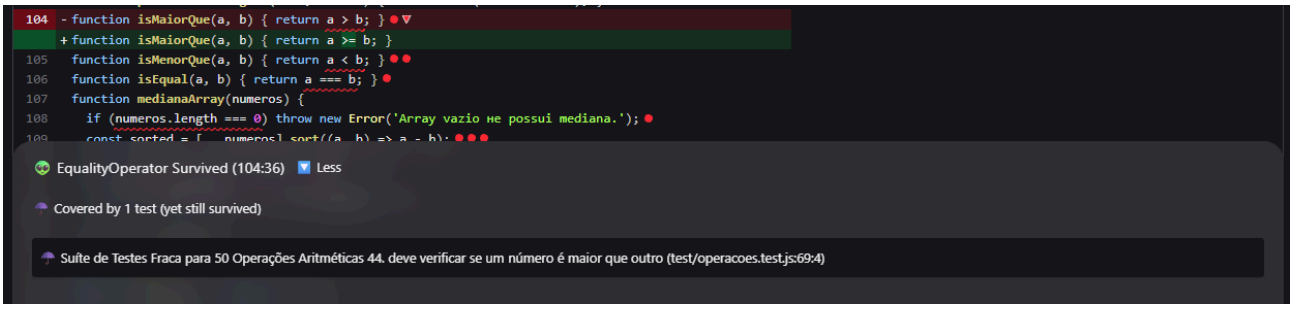


Figura 4 - Mutante sobrevivente em isMaiorQue.



3. Solução implementada

Acrescentei 22 testes, agrupados em describe por função, **sem mexer no código de** src/. A ideia foi sempre perguntar "que entrada faria o mutante dar um resultado diferente do original?". Os novos casos cobrem fronteiras, resultados false, números negativos, arrays vazios, conversões de temperatura com valores que não coincidem por acaso (zero dava 32 nas versões certa e errada) e mensagens de erro exatas. Alguns exemplos:

```
describe('isPrimo', () => {
 test('deve tratar um e um composto como nao primos', () => {
 expect(isPrimo(1)).toBe(false);
 expect(isPrimo(4)).toBe(false);
 });
});

describe('isMaiorQue', () => {
 test('deve retornar falso para menores e iguais', () => {
 expect(isMaiorQue(4, 5)).toBe(false);
 expect(isMaiorQue(5, 5)).toBe(false);
 });
});
```

isPrimo(4) obriga o programa a entrar no ramo que acha um divisor, o que mata a remoção do bloco. isImpar(2) separa o resto da multiplicação, já que 2 * 2 !== 0 daria true no mutante e o esperado é false. E isMaiorQue(5, 5) é o teste de fronteira que derruba o >=. Com o código original, os 72 testes (50 antigos + 22 novos) passam.

Mutantes equivalentes. Sete sobreviventes não podem ser mortos por nenhum teste no domínio válido: quatro

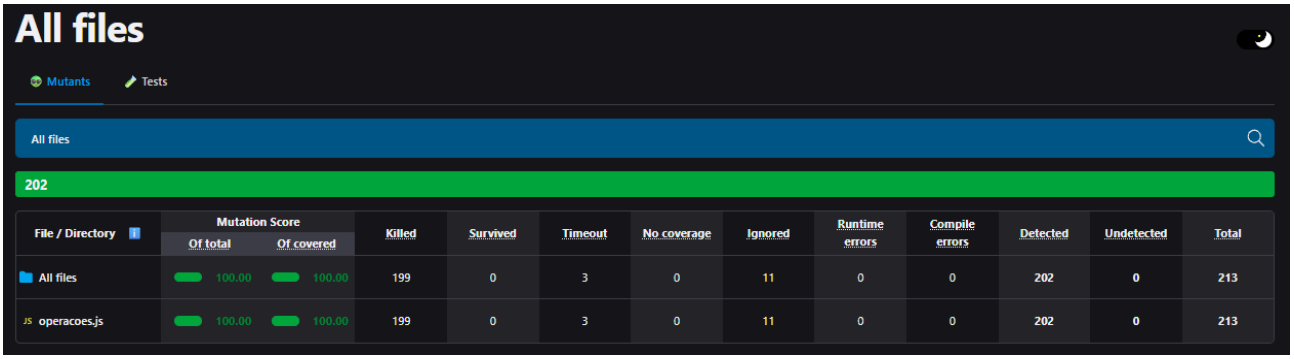
variantes da condição-base do fatorial (0 e 1 já retornam 1 pelo laço), a checagem de array vazio em produtoArray (a semente 1 do reduce já resolve) e as trocas < por <= e > por >= do clamp (no limite, o retorno é o próprio limite). Em vez de criar testes artificiais, usei as diretivas oficiais do Stryker com a justificativa em comentário. Como elas valem por linha e operador, também acabaram ignorando quatro variantes que, sem as diretivas, estavam sendo mortas. Deixo isso explícito aqui em vez de chamá-las de equivalentes.

4. Resultados finais

Métrica	Inicial	Final
Cobertura statements / branches / functions / lines	85,41% / 58,82% / 100% / 98,64%	100% / 100% / 100% / 100%
Mutation score (Stryker)	73,71%	100%
Mutantes gerados	213	213
Mortos / timeouts	154 / 3	199 / 3
Sobreviventes / sem cobertura	44 / 12	0 / 0
Ignorados (diretivas)	0	11

O Stryker mostra 100% porque conta os 3 timeouts como mutantes detectados. Para uma conta mais conservadora, que considera só os mutantes mortos e tira os 11 ignorados do total, o resultado é **199 / (213 - 11) = 98,51%**, ainda acima da meta de 98%.

Figura 5 - Resumo do relatório final do Stryker.



5. Conclusão

O trabalho deixou claro que cobertura alta não prova que a suíte é boa. A inicial executava quase todas as linhas, mas deixava passar muitos defeitos plausíveis por falta de asserções que discriminam e de casos de fronteira. O teste de mutação mostrou exatamente onde estavam essas brechas e serviu de guia para escrever testes que realmente conferem o comportamento.

Também ficam algumas limitações. Mutantes equivalentes não têm teste que os mate, então precisam ser justificados em vez de forçados. O Stryker gera sobretudo mutantes de primeira ordem (um defeito por vez); defeitos combinados (de ordem superior) podem se mascarar e são mais difíceis de achar. E há o custo de execução: aqui foram 213 mutantes em cerca de sete segundos, mas em projetos grandes o tempo cresce bastante.